

gem5 Cache Compression Function Documentation

1. Simulation Setup & Processor Control Module

1.1 Processor Configuration & KVM Acceleration

ConfigureProcessor(isa, numCores, cpuType)

Input

- isa: ISA (x86_64)
- numCores: int (1)
- cpuType: CPUTypes (KVM / DerivO3CPU)

Output

- processor: SimpleSwitchableProcessor
-

SetupKvmFastBoot(processor, usePerf)

Input

- processor: SimpleSwitchableProcessor
- usePerf: bool (False - Cloud Nested VM Fix)

Output

- kvmCoreConfig: X86KvmCPU
 - pmuStatus: Disabled (prevents PerfKvmCounter::attach panic)
-

SwitchToDerivO3CPU(simulator, warmupInsts, maxInsts)

Input

- simulator: Simulator
- warmupInsts: uint64_t (50,000,000)
- maxInsts: uint64_t (500,000,000 / 200,000,000)

Output

- exitEvent: ExitEvent.MAX_INSTS
 - switchStatus: CPU_SWITCHED_TO_O3
-

ResetSimulationStatistics()

Input

- None (Invoked after warmup instruction threshold)

Output

- statsResetStatus: STATS_CLEARED_FOR_ROI
-

DumpSimulationStatistics(statsFile, dumpPhase)

Input

- statsFile: Path (stats.txt)
- dumpPhase: Enum (BOOT_DUMP_1 / ROI_MEASURED_DUMP_2)

Output

- simulationStatsRecord: SimulationROIStats
-

1.2 Storage, Kernel & OS Boot Environment

MountDiskImage(diskResource, rootPartition)

Input

- diskResource: DiskImageResource (spec2017.img)
- rootPartition: str (1 -> /dev/hda1)

Output

- mountedStorage: DiskImage
-

LoadLinuxKernel(kernelResource, bootArgs)

Input

- kernelResource: KernelResource (vmlinux-5.4.0)
- bootArgs: str (earlyprintk=ttyS0 console=ttyS0 lpj=7999923)

Output

- bootContext: LinuxX86BootLoader
-

ExecuteBenchmarkCommand(workloadString, m5ExitTrigger)

Input

- workloadString: str (spec2017 505.mcf_r / 541.leela_r)
- m5ExitTrigger: bool (True)

Output

- executionExitCode: int (0: Success)
-

2. Cache Hierarchy & Interconnect Module

2.1 Cache Subsystem Configuration

CreateL1ICache(size, assoc, hitLatency)

Input

- size: MemorySize ('32KiB')
- assoc: int (8)
- hitLatency: Cycles (1)

Output

- l1iCache: L1ICache
-

CreateL1DCache(size, assoc, hitLatency)

Input

- size: MemorySize ('32KiB')
- assoc: int (8)
- hitLatency: Cycles (1)

Output

- l1dCache: L1DCache
-

CreateL2Cache(size, assoc, compressorChoice)

Input

- size: MemorySize ('256KiB' / '512KiB')
- assoc: int (16)
- compressorChoice: str
 - 'none' -> Standard L2 Cache (No Compression)
 - 'bdi' -> BDI MultiCompressor + CompressedTags
 - 'cpack' -> CPack Compressor + CompressedTags
 - 'fpc' -> FPC Compressor + CompressedTags
 - 'zero' -> ZeroCompressor + CompressedTags

Output

- l2Cache: L2Cache
 - tags: CompressedTags
 - compressor: BaseCacheCompressor
-

SetupIOCache(board, membus)

Input

- board: X86Board
- membus: SystemXBar

Output

- iocache: Cache (1KiB, 8-way coherent I/O bridge)
-

2.2 Crossbar & Port Binding

IncorporateCache(board, membus)

Input

- board: X86Board
- membus: SystemXBar

Output

- hierarchyStatus: CACHES_CONNECTED
 - l2XBars: L2XBar[]
-

ConnectSystemMemoryBus(l2Cache, ddrController)

Input

- l2Cache: L2Cache
- ddrController: DualChannelDDR4_2400

Output

- memoryPortBinding: MemSidePort
-

3. Tag Store & Superblock Co-Allocation Module

3.1 Tag Consistency & Move Operations (Core Bug Fix)

CopyTagsFrom(sourceEntry)

Input

- sourceEntry: TaggedEntry (Extracted Tag + Security State)

Output

- destinationEntry: TaggedEntry (Tag copied directly without re-extraction)

SectorSubBlkMove(destinationBlk, sourceBlk)

Input

- destinationBlk: SectorSubBlk
- sourceBlk: SectorSubBlk

Output

- updatedSectorBlk: SectorSubBlk
- panicStatus: NONE (Eliminates 'Overwriting valid sector!' panic)

CompressionBlkMove(destinationBlk, sourceBlk)

Input

- destinationBlk: CompressionBlk
- sourceBlk: CompressionBlk

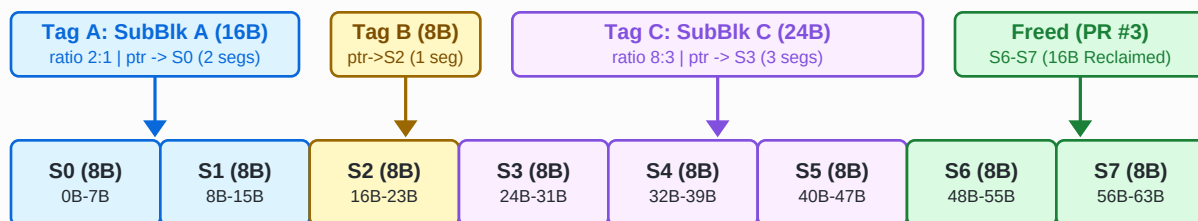
Output

- updatedCompressionBlk: CompressionBlk (Transfers compression size & factor)

3.2 Superblock Co-Allocation & Compaction

Figure 3.1: Superblock Physical Storage & Dynamic Sector Co-Allocation (PR #3)

Physical 64-Byte Cache Line partitioned into 8-Byte Segments (S0..S7) with Multi-Tag Pointers



SectorSubBlk Co-Allocation:

Up to 4 sub-blocks share one physical 64B line, saving tag ways.

The Bug Fixed (copyTagsFrom):

Direct tag transfer eliminates 'Overwriting valid sector!' panic on migration.

PR #3 Dynamic Compaction:

updateCompressionFactor() recalculates bounds upon eviction, freeing S6-S7.

UpdateCompressionFactor(superBlk) [PR #3]

Input

- superBlk: SuperBlk

Output

- newCompressionFactor: uint8_t (Recalculated minimum size among valid sub-blocks)
 - reclaimedPhysicalWays: int (Freed for new co-allocations)
-

CoAllocateSubBlock(superBlk, subBlk, requiredSectors)

Input

- superBlk: SuperBlk
- subBlk: SectorSubBlk
- requiredSectors: int (1 to 4)

Output

- allocationStatus: CO_ALLOCATED_SUCCESS
 - coAllocatedSubBlkCount: int (Up to 4 sub-blocks in 64B line)
-

HandleDataExpansion(cacheBlk, newCompressedSize)

Input

- cacheBlk: CacheBlk
- newCompressedSize: int (Bytes after dirty write modification)

Output

- expansionStatus: EXPANSION_HANDLED
 - migratedWayIndex: int (111,926 events in 505.mcf_r)
-

HandleDataContraction(cacheBlk, newCompressedSize)

Input

- cacheBlk: CacheBlk
- newCompressedSize: int (Bytes shrunk after write)

Output

- contractionStatus: CONTRACTION_HANDLED
 - freedSectorSlots: int (16,790 events in 505.mcf_r)
-

InvalidateSubBlock(subBlk)

Input

- subBlk: SectorSubBlk

Output

- invalidationStatus: SUB_BLOCK_INVALIDATED
 - triggeredFactorUpdate: SuperBlk::updateCompressionFactor()
-

4. MultiCompressor Coordination & Arbitration Module

4.1 Parallel Trial Compressions & Arbitration

InitializeMultiCompressor(compressors[], encodingInTags)

Input

- compressors: BaseCacheCompressor[]
 - ZeroCompressor (size_threshold: 99%)
 - RepeatedQwordsCompressor (size_threshold: 99%)
 - Base64Delta8 (size_threshold: 99%)
 - Base64Delta16 (size_threshold: 99%)
 - Base64Delta32 (size_threshold: 99%)
 - Base32Delta8 (size_threshold: 99%)
 - Base32Delta16 (size_threshold: 99%)
 - Base16Delta8 (size_threshold: 99%)
- encodingInTags: bool (True)

Output

- multiCompressorInstance: MultiCompressor
-

EvaluateParallelCompressions(rawBlock512)

Input

- rawBlock512: uint64_t[8] (512-bit cache line data)

Output

- compressionResults: CompressionData[8]
 - trialLatencies: Cycles[8]
-

RankSubCompressors(compressedSizes[])

Input

- compressedSizes: int[8] (Size in bits from each sub-engine)

Output

- ranks: int[8] (Ranks 0 through 7 for smallest footprint)
-

SelectWinningCompressor(ranks[])

Input

- ranks: int[8] (Rank vector)

Output

- winningCompressorID: int (0 to 7)
 - finalCompressedSizeBits: int (0, 64, 128, 160, 192, 256, 288, 320, or 512)
-

4.2 Dynamic Failure Tracking & Probing [PR #2]

TrackSubCompressorFailure(compressorID, isFailure)

Input

- compressorID: int (0 to 7)
- isFailure: bool (True if compressed_size >= threshold)

Output

- consecutiveFailureCount: uint32_t
 - skippedState: bool (True if count >= unpromising_threshold 100)
-

ProbeSkippedCompressor(compressorID, totalCompressions)

Input

- compressorID: int
- totalCompressions: uint64_t
- probeInterval: uint32_t (1,000)

Output

- probeTriggered: bool (Enables single periodic trial pass)

DecodeWinningSubCompressor(tagBits)

Input

- tagBits: TagMetadata (Sub-compressor ID stored in tag)

Output

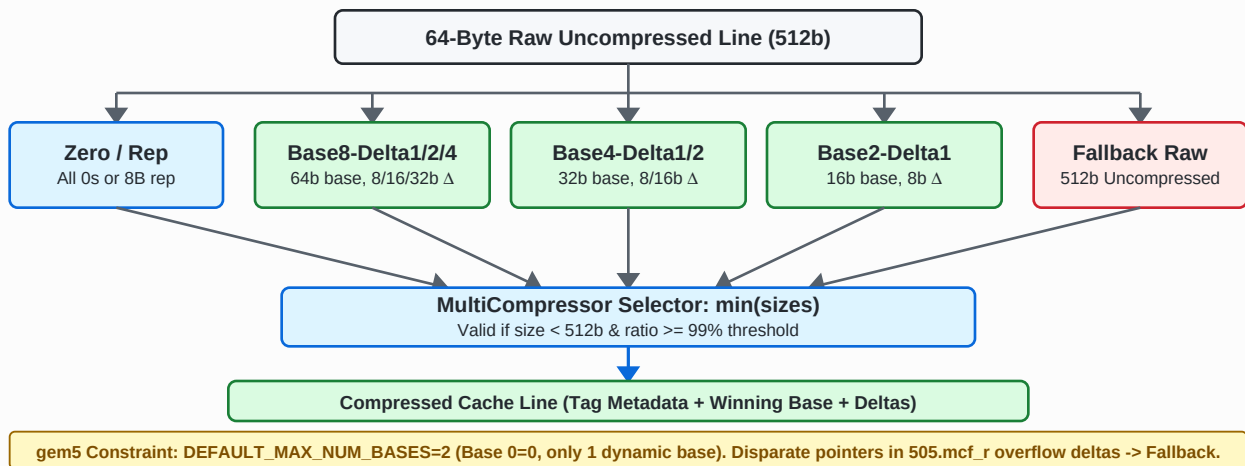
- subCompressorEngine: BaseCacheCompressor

5. Base-Delta-Immediate (BDI) Sub-Engine Module

5.1 Dictionary & Base Management

Figure 5.1: BDI Parallel Encoders & Selection Pipeline

Parallel trial evaluation across Base-Delta configurations with gem5 2-Base Constraint



ResetDictionary(dictionary)

Input

- dictionary: DictionaryEntry[] (Capacity: 2 entries)

Output

- initializedDictionary: DictionaryEntry[]
- immediateBase0: 0x0000000000000000 (Reserved zero base)

AddToDictionary(entryData)

Input

- entryData: uint64_t / uint32_t / uint16_t (First unmatched word)

Output

- allocatedBase1: DictionaryEntry (The single allowed dynamic base)
 - numEntries: int (Must not exceed DEFAULT_MAX_NUM_BASES = 2)
-

CompressValue(chunkValue, dictionary[])

Input

- chunkValue: BaseType
- dictionary: DictionaryEntry[]

Output

- matchedPattern: Pattern (PatternM: Match / PatternX: No-Match Uncompressed)
-

5.2 Individual Base-Delta Encoders

CompressZeroBlock(chunks[])

Input

- chunks: uint64_t[8]

Output

- sizeBits: 0 bits
 - compressionRatio: Infinite (0 bits data payload)
-

CompressRepeatedQwords(chunks[])

Input

- chunks: uint64_t[8]

Output

- sizeBits: 64 bits (8:1 compression)
 - storedValue: uint64_t (Single repeated quadword)
-

CompressBase64Delta8(chunks[])

Input

- chunks: uint64_t[8]

Output

- sizeBits: 128 bits (4:1 compression)
 - base: 1 x 64-bit quadword
 - deltas: 8 x 8-bit signed deltas (-128 to +127)
-

CompressBase64Delta16(chunks[])

Input

- chunks: uint64_t[8]

Output

- sizeBits: 192 bits (8:3 compression)
 - base: 1 x 64-bit quadword
 - deltas: 8 x 16-bit signed deltas (-32768 to +32767)
-

CompressBase64Delta32(chunks[])

Input

- chunks: uint64_t[8]

Output

- sizeBits: 320 bits (8:5 compression)
 - base: 1 x 64-bit quadword
 - deltas: 8 x 32-bit signed deltas
-

CompressBase32Delta8(chunks[])

Input

- chunks: uint32_t[16]

Output

- sizeBits: 160 bits (16:5 compression)
 - base: 1 x 32-bit doubleword
 - deltas: 16 x 8-bit signed deltas
-

CompressBase32Delta16(chunks[])

Input

- chunks: uint32_t[16]

Output

- sizeBits: 288 bits (16:9 compression)
 - base: 1 x 32-bit doubleword
 - deltas: 16 x 16-bit signed deltas
-

CompressBase16Delta8(chunks[])

Input

- chunks: uint16_t[32]

Output

- sizeBits: 272 bits (32:17 compression)
 - base: 1 x 16-bit word
 - deltas: 32 x 8-bit signed deltas
-

DecompressBaseDelta(base, deltas[], numChunks)

Input

- base: BaseType
- deltas: DeltaType[]
- numChunks: int

Output

- reconstructedChunks: BaseType[] (Recomputed as $\text{chunk}[i] = \text{base} + \text{delta}[i]$)
 - decompLatency: Cycles (1 cycle nominal)
-

6. Dictionary & Pattern Compression Module (C-Pack & FPC)

6.1 C-Pack Word-Level Pattern Matching

MatchPatternZZZZ(word32)

Input

- word32: uint32_t (Word = 0x00000000)

Output

- matchedCode: '00' (2 bits payload)
 - allocatedDictionary: False
-

MatchPatternXXXX(word32)

Input

- word32: uint32_t (No match in 16-entry dictionary)

Output

- matchedCode: '01' + 32-bit literal (34 bits total)
 - allocatedDictionary: True (Inserts word into dictionary)
-

MatchPatternMMMM(word32, dictionary[])

Input

- word32: uint32_t (Full match with dictionary[i])

Output

- matchedCode: '10' + 4-bit index (6 bits total)
 - pointerPrefixCompacted: True (Compacts 64-bit 0x00007FFF prefix)
-

MatchPatternMMXX(word32, dictionary[])

Input

- word32: uint32_t (High 2 bytes match dictionary[i], low 2 bytes new)

Output

- matchedCode: '11' + 4-bit index + 16-bit literal (22 bits total)
-

MatchPatternZZZX(word32)

Input

- word32: uint32_t (High 3 bytes zero, low 1 byte new)

Output

- matchedCode: '1100' + 8-bit literal (12 bits total)
-

MatchPatternMMMX(word32, dictionary[])

Input

- word32: uint32_t (High 3 bytes match dictionary[i], low 1 byte new)

Output

- matchedCode: '1101' + 4-bit index + 8-bit literal (16 bits total)
-

6.2 Fast Path & Zero Decompression [PR #4]

FastZeroDecompress(compData, outputBuffer) [PR #4]

Input

- compData: CompressionData (isAllZero() == True)
- outputBuffer: uint8_t[64]

Output

- outputBuffer: Zeroed memory via std::memset()
- decompLatency: Cycles(1) (Bypasses 8-16 cycle chunk iteration)

MatchFPCZeroRun(runCount)

Input

- runCount: int (Consecutive zero words, up to 8)

Output

- fpcCode: '000' + 3-bit run count (6 bits total)

MatchFPCSignExtended(word32, bitWidth)

Input

- word32: uint32_t
- bitWidth: int (4, 8, or 16 bits)

Output

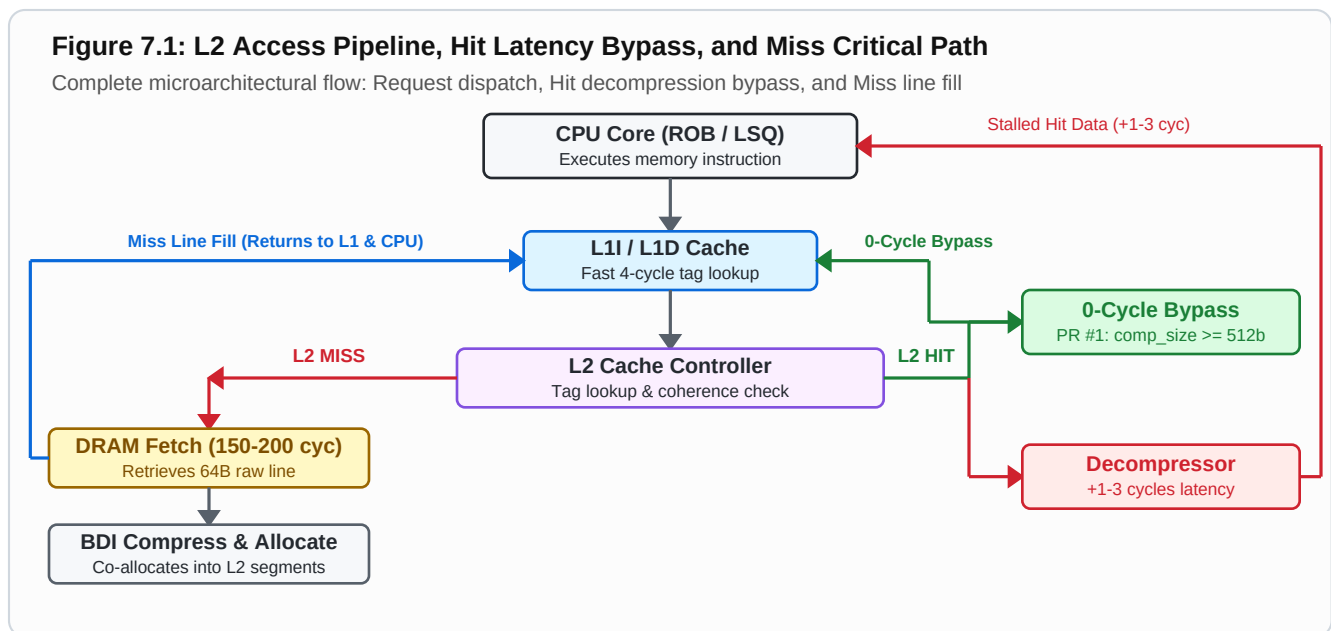
- fpcCode: Prefix + Truncated literal (7, 11, or 19 bits)

7. Latency Modeling & Adaptive Filtering Module

7.1 Decompression Latency Bypass [PR #1 & #5]

Figure 7.1: L2 Access Pipeline, Hit Latency Bypass, and Miss Critical Path

Complete microarchitectural flow: Request dispatch, Hit decompression bypass, and Miss line fill



CalculateDecompressionLatency(compressedSizeBits, blockSizeBits, baseLatency)

Input

- compressedSizeBits: std::size_t
- blockSizeBits: std::size_t (512 bits)
- baseLatency: Cycles (1 to 3 cycles)

Output

- decompLatency: Cycles (Calculated decompressor pipeline latency)
-

ApplyUncompressedLatencyBypass(compressedSizeBits, blockSizeBits) [PR #1 & #5]

Input

- compressedSizeBits: std::size_t
- blockSizeBits: std::size_t (512 bits)

Output

- decompLatency: Cycles(0) (If comp_size >= 512 bits, bypasses decompression delay entirely)
 - pipelineStallsAvoided: 2,061,129 cycles on 505.mcf_r
-

7.2 Adaptive Breakeven Filtering [PR #6]

CalculateBreakevenThreshold(l2Hits, dramLatency, nominalDecompLatency)

Input

- l2Hits: uint64_t (13,759,243)
- dramLatency: Cycles (150 cycles)
- nominalDecompLatency: Cycles (1 cycle)

Output

- breakevenMissSavings: uint64_t (Delta Misses needed: 91,728)
-

MonitorMovingAverageCompressionRatio(windowSize, observedRatio)

Input

- windowSize: uint32_t (10,000 blocks)
- observedRatio: float (1.09x on 505.mcf_r / 1.45x on 541.leela_r)

Output

- adaptiveState: Enum (COMPRESS_ENABLED / COMPRESS_BYPASSED)
-

ApplyAdaptiveBypassFilter(observedRatio, threshold) [PR #6]

Input

- observedRatio: float
- threshold: float (1.20x)

Output

- bypassTriggered: bool (Treats blocks as uncompressed when compression gains < latency costs)
-

8. Experiment Sweep & Metrics Extraction Module

8.1 Sweep Orchestration Runner (run_spec2017_compression_sweep.sh)

ParseExperimentArguments(benchmark, l2Size, compressor)

Input

- benchmark: str
 - 541.leela_r
 - 502.gcc_r
 - 538.imagick_r
 - 500.perlbench_r
- l2Size: str
 - 256KiB
 - 512KiB
 - 1024KiB
- compressor: str
 - none (baseline)
 - bdi
 - cpack
 - fpc
 - zero

Output

- experimentConfig: SweepParameters
 - outputDirectory: Path (m5out_eval_sweep/___)
-

ExecuteSimulationRun(outdir, configScript, parameters)

Input

- outdir: Path
- configScript: Path (configs/spec2017_compression_kvm.py)
- parameters: SweepParameters

Output

- simulationExitCode: int (0: Completed)
 - simRunLog: File (outdir/sim_run.log)
-

8.2 Empirical Metric Extraction

ExtractCoreIPC(statsFilePath)

Input

- statsFilePath: Path (m5out/stats.txt, Dump #2)

Output

- ipc: float (board.processor.switch.core.ipc)
 - cpi: float (board.processor.switch.core.cpi)
 - simCycles: uint64_t (simCycles)
-

ExtractCompressionDistribution(statsFilePath)

Input

- statsFilePath: Path

Output

- compressionDistribution: Dictionary
 - compressionSize::0 (Zero Blocks)
 - compressionSize::64 (8:1 Blocks)
 - compressionSize::256 (2:1 Blocks)
 - compressionSize::512 (Incompressible Blocks)
 - avgCompressionSizeBits: float
 - compressions / failedCompressions: uint64_t
-

ExtractTagStoreEvents(statsFilePath)

Input

- statsFilePath: Path

Output

- dataExpansions: uint64_t
 - dataContractions: uint64_t
 - l2Replacements: uint64_t
-

ExtractDRAMTrafficMetrics(statsFilePath)

Input

- statsFilePath: Path

Output

- dramBytesRead: uint64_t
 - dramBytesWritten: uint64_t
 - totalDramTraffic: uint64_t
-

9. Data Structure & Memory Layout Specifications

9.1 Workload Memory Structures (SPEC CPU2017 505.mcf_r)

NodeStructure (node_t)

Input

- Memory Layout: Contiguous 64-byte heap element

Output

- Cache Footprint: 8 x 64-bit heterogeneous words

```
node_t {
    int64_t potential; // Dual variable / node potential (8 bytes)
    int64_t orientation; // Simplex basis tree direction (+1 / -1) (8 bytes)
    struct node *child; // Pointer to first child node (8 bytes)
    struct node *sibling; // Pointer to next sibling node (8 bytes)
    struct node *parent; // Pointer to parent node in basis tree (8 bytes)
    struct arc *pred; // Pointer to predecessor arc (8 bytes)
    int64_t time; // Schedule timestamp (8 bytes)
    int64_t number; // Node index (8 bytes)
} // sizeof(node_t) = 64 bytes (Matches exact L2 cache line size)
```

ArcStructure (arc_t)

Input

- Memory Layout: Arc array element

Output

- Cache Footprint: 6 x 64-bit words

```
arc_t {
    int64_t cost; // Network flow cost (8 bytes)
    struct node *tail; // Source node pointer (8 bytes)
    struct node *head; // Destination node pointer (8 bytes)
    int64_t ident; // Arc identifier (8 bytes)
    struct arc *nextout; // Pointer to next outgoing arc (8 bytes)
    int64_t flow; // Current flow variable (8 bytes)
} // sizeof(arc_t) = 48 bytes (Spans across cache line boundaries)
```

9.2 gem5 Architectural & Tracking Descriptors

SuperBlkDescriptor

Input

- Physical Storage Container: 64-byte way

Output

- Co-allocation Tracking: Sector bounds

```
SuperBlkDescriptor {  
    uint32_t physicalWayIndex;  
    uint8_t compressionFactor; // Managed by updateCompressionFactor()  
    std::vector subBlks; // Max 4 sub-blocks per superblock  
    bool isValid;  
    Addr superblockTag;  
}
```

SectorSubBlkDescriptor

Input

- Architectural Cache Line Tag Entry

Output

- Metadata: State & compression attributes

```
SectorSubBlkDescriptor {  
    Addr tag; // Extracted address tag  
    bool isSecure; // Security attribute  
    bool isValid; // Coherence valid bit  
    bool isModified; // Dirty state  
    uint8_t subBlockIndex; // Index within parent superblock (0 to 3)  
    uint16_t compressedSizeBits; // Physical bits occupied in data array  
}
```

SimulationROIStats

Input

- Simulation Dump 2 Data Structure

Output

- Complete Processor & Cache Metrics

```
SimulationROIStats {  
    uint64_t committedInstructions; // 200,000,000 / 500,000,000  
    uint64_t simulatedCycles; // Exact cycle count  
    double ipc; // Instructions Per Cycle  
    double cpi; // Cycles Per Instruction  
    uint64_t l2DemandAccesses; // Total L2 demand references  
    uint64_t l2DemandMisses; // Unserved L2 misses  
    uint64_t l2Replacements; // Evicted cache blocks  
    uint64_t dataExpansions; // Dynamic block growth relocations  
    uint64_t dataContractions; // Dynamic block shrinkage releases  
    uint64_t dramBytesRead; // Main memory read volume  
    uint64_t dramBytesWritten; // Main memory writeback volume  
}
```
